Fast-DDPM: Code Notes

Fast Denoising Diffusion Probabilistic Models for Medical Image-to-Image Generation

ECE 551 project | Luis Carrillo

Code: github.com/mirthAI/Fast-DDPM | Run on UNM Easley (L40S, PyTorch 2.2.2)

Draft. Code is quoted from the repository; slides marked DRAFT depend on experiments not yet run.

References

- **Paper.** H. Jiang, M. Imran, T. Zhang, Y. Zhou, M. Liang, K. Gong, W. Shao, “Fast-DDPM: Fast Denoising Diffusion Probabilistic Models for Medical Image-to-Image Generation,” IEEE JBHI, 2025. arXiv:2405.14802
- **Code.** github.com/mirthAI/Fast-DDPM (adapted by the authors from the DDIM codebase, github.com/ermongroup/ddim)
- **Sampler.** J. Song, C. Meng, S. Ermon, “Denoising Diffusion Implicit Models,” 2020 (arXiv:2010.02502)
- **Base method.** J. Ho, A. Jain, P. Abbeel, “Denoising Diffusion Probabilistic Models,” NeurIPS 2020
- **Textbook.** A. Torralba, P. Isola, W. T. Freeman, *Foundations of Computer Vision*, MIT Press, 2024 (visionbook.mit.edu), diffusion chapter: *chapter/equation numbers: empty*
- **Index of models and datasets.** github.com/pattichis/AIM
- **Format reference and 3D example.** M. S. Pattichis, MLSP 2026 tutorial, github.com/pattichis/MLSP2026

Equations on these slides are written out directly; nothing is copied from the book. Long code lines are wrapped for display; the logic is unchanged.

The task: conditional image-to-image generation

Given a **condition** image c, generate the **target** image x_0.

Task	Condition c	Target x_0	Dataset	--dataset
Denoising	low-dose CT slice	full-dose CT slice	LDCT-and-Projection-data	`LDFDCT`
Translation	T1-weighted MRI	T2-weighted MRI	BraTS 2018	`BRATS`
Super-resolution	previous + next MRI slice	middle MRI slice	Prostate-MRI-US-Biopsy	`PMUB`

- The model is **2D**: it processes one 256×256 slice at a time.
- These slides follow **LDFDCT** (CT denoising), the task we reproduced.
- Same machinery for all three; they differ in how many condition channels go in (1 or 2).

paper Sec. IV-A; README.md; runners/diffusion.py

Diffusion in two phases

Training

Pair (c, x_0) → Pick a time step t → Jump to noisy x_t → U-Net predicts the noise ε from (c, x_t, t) → Squared error; update weights

Deployment (sampling)

Condition c + random noise x_T → U-Net predicts ε → DDIM step to the next chosen t → repeat ~10 times → Generated target $\hat{x}_0$

> Fast-DDPM's change: **train and sample on the same ~10 time steps**, instead of training on 1,000 and sampling on a subset.

paper Sec. III, Algorithms 1 and 2

Training data: a pair of images

```
class LDFDCT(Dataset):
    def __init__(self, dataroot, img_size, split='train', data_len=-1):
        self.img_ld_path, self.img_fd_path = \
            get_paths_from_images(dataroot)   (1)

    def __getitem__(self, index):
        img_LD = Image.open(self.img_ld_path[index]).convert("L")   (2)
        img_FD = Image.open(self.img_fd_path[index]).convert("L")
        img_LD = img_LD.resize((self.img_size, self.img_size))   (3)
        img_FD = img_FD.resize((self.img_size, self.img_size))
        [img_LD, img_FD] = transform_augment(
            [img_LD, img_FD], split=self.split, min_max=(-1, 1))   (4)
        return {'FD': img_FD, 'LD': img_LD, 'case_name': case_name}   (5)
```

1. Finds `*ld.png` and `*fd.png` files and sorts both lists, so index *i* in each is the same slice.
2. Grayscale, one channel.
3. Resized to 256×256 (`image_size` in the config).
4. Scaled to [−1, 1]; training also applies a random horizontal flip to the pair.
5. Keys `'LD'` (condition) and `'FD'` (target) are hard-coded in the training loop.

datasets/LDFDCT.py, datasets/sr_util.py

The noise schedule

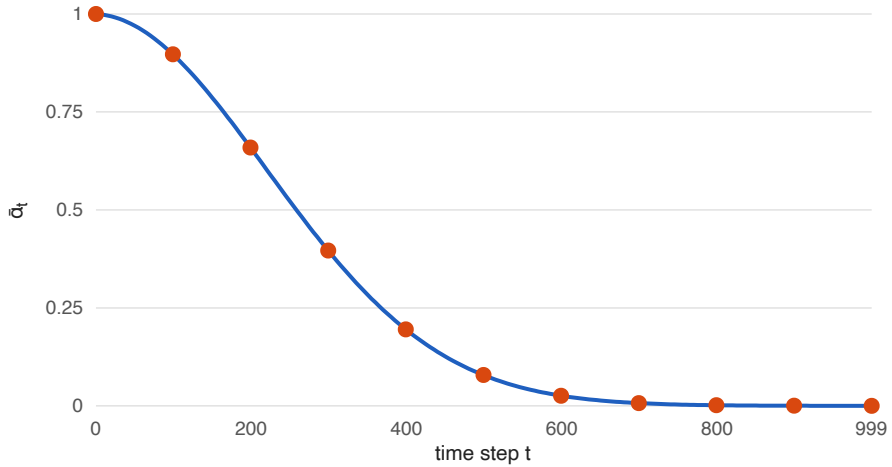
$$\beta_t \text{ rises linearly from } 10^{-4} \text{ to } 0.02, \quad t = 1 \ \ldots \ 1000$$

$$\bar{\alpha}_t = \prod_{s \le t} (1 - \beta_s)$$

```
betas = np.linspace(beta_start, beta_end,
                    num_diffusion_timesteps)   (1)
alphas = 1.0 - betas
alphas_cumprod = alphas.cumprod(dim=0)   (2)
```

(1) Linear β schedule (config: 0.0001 → 0.02, 1000 steps).

(2) **$\bar{\alpha}_t$**: the fraction of the original signal left at step t. The paper writes it $\alpha^2(t)$.



$\bar{\alpha}_t$ for the real schedule (computed). Orange dots: the 11 time steps used by the **uniform** set. By $t \approx 673$, $\bar{\alpha} < 0.01$: the image is almost pure noise.

The forward process in one jump

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon, \quad \varepsilon \sim N(0, I)$$

```
def sg_noise_estimation_loss(model, x_img, x_gt, t, e, b,
                             keepdim=False):
    a = (1-b).cumprod(dim=0).index_select(0, t).view(-1, 1, 1, 1)  (1)
    x = x_gt * a.sqrt() + e * (1.0 - a).sqrt()  (2)
```

1. `a` is $\bar{\alpha}_t$ for each image in the batch (one t per image), reshaped to broadcast over pixels.
2. The noisy **target** `x`. Note it noises `x_gt` (the full-dose image), not the condition. `e` is ε.

No loop over t: because every step is Gaussian, any noise level is reachable directly from x_0. This is why training can pick any t cheaply.

Fast-DDPM's change: which time steps?

```
if self.args.scheduler_type == 'uniform':
    skip = self.num_timesteps // self.args.timesteps   (1)
    t_intervals = torch.arange(-1, self.num_timesteps, skip)
    t_intervals[0] = 0   (2)
elif self.args.scheduler_type == 'non-uniform':
    t_intervals = torch.tensor(
        [0, 199, 399, 599, 699, 799, 849, 899, 949, 999])   (3)
```

1. 1000 // 10 = 100, so every 100th step.
2. `arange(-1, 1000, 100)` gives –1, 99, …, 999; setting the first to 0 leaves **11** values (see “To verify”).
3. Non-uniform: more points at high noise (t ≥ 699). Paper: better for super-resolution; uniform better for denoising/translation.

t	$\bar{\alpha}_t$	signal $\sqrt{\bar{\alpha}}$	noise $\sqrt{(1-\bar{\alpha})}$
0	0.9999	1.000	0.010
99	0.8970	0.947	0.321
199	0.6590	0.812	0.584
299	0.3964	0.630	0.777
399	0.1951	0.442	0.897
499	0.0786	0.280	0.960
599	0.0259	0.161	0.987
699	0.0070	0.083	0.997
799	0.0015	0.039	0.999
899	0.0003	0.017	1.000
999	0.0000	0.006	1.000

Uniform set, computed from the schedule. Training and sampling both use *only* these noise levels.

Picking t for each training batch

```
idx_1 = torch.randint(0, len(t_intervals), size=(n // 2 + 1,))   1
idx_2 = len(t_intervals) - idx_1 - 1   2
idx = torch.cat([idx_1, idx_2], dim=0)[:n]   3
t = t_intervals[idx].to(self.device)   4

loss = loss_registry[config.model.type](
    model, x_img, x_gt, t, e, b)   5
```

1. Random indices into the small set of chosen time steps.
2. Each index is paired with its mirror (first ↔ last, …): if one image gets a low-noise step, another gets a high-noise step.
3. Concatenate and trim to the batch size *n*.
4. Map indices to actual time steps.
5. `config.model.type` = `'sg'` selects the loss on the previous slide.

> This is *antithetic sampling*: it balances noise levels inside every batch and lowers gradient variance. The paper does not mention it; it is in the code.

Telling the network which t it is seeing

$$emb(t) = [\, \sin(t\cdot\omega_1), \ldots, \sin(t\cdot\omega_k), \cos(t\cdot\omega_1), \ldots, \cos(t\cdot\omega_k) \,], \quad \omega_j = 10000^{-j/(k-1)}$$

```
half_dim = embedding_dim // 2
emb = math.log(10000) / (half_dim - 1)
emb = torch.exp(
    torch.arange(half_dim, dtype=torch.float32) * -emb)   1
emb = timesteps.float()[:, None] * emb[None, :]   2
emb = torch.cat([torch.sin(emb), torch.cos(emb)], dim=1)   3

# in Model.forward
temb = get_timestep_embedding(t, self.ch)
temb = self.temb.dense[0](temb)   4
temb = nonlinearity(temb)
temb = self.temb.dense[1](temb)
```

1. A ladder of frequencies from 1 down to 1/10000.
2. Multiply each time step by every frequency.
3. Sine and cosine of those products: a unique vector for each t (the same idea as positional encoding in transformers).
4. Two linear layers expand it to width 4·ch (512).

Residual block: where t enters

```
def forward(self, x, temb):
    h = x
    h = self.norm1(h)
    h = nonlinearity(h)                     # SiLU: x * sigmoid(x)
    h = self.conv1(h)

    h = h + self.temb_proj(nonlinearity(temb))[:, :, None, None]   (1)

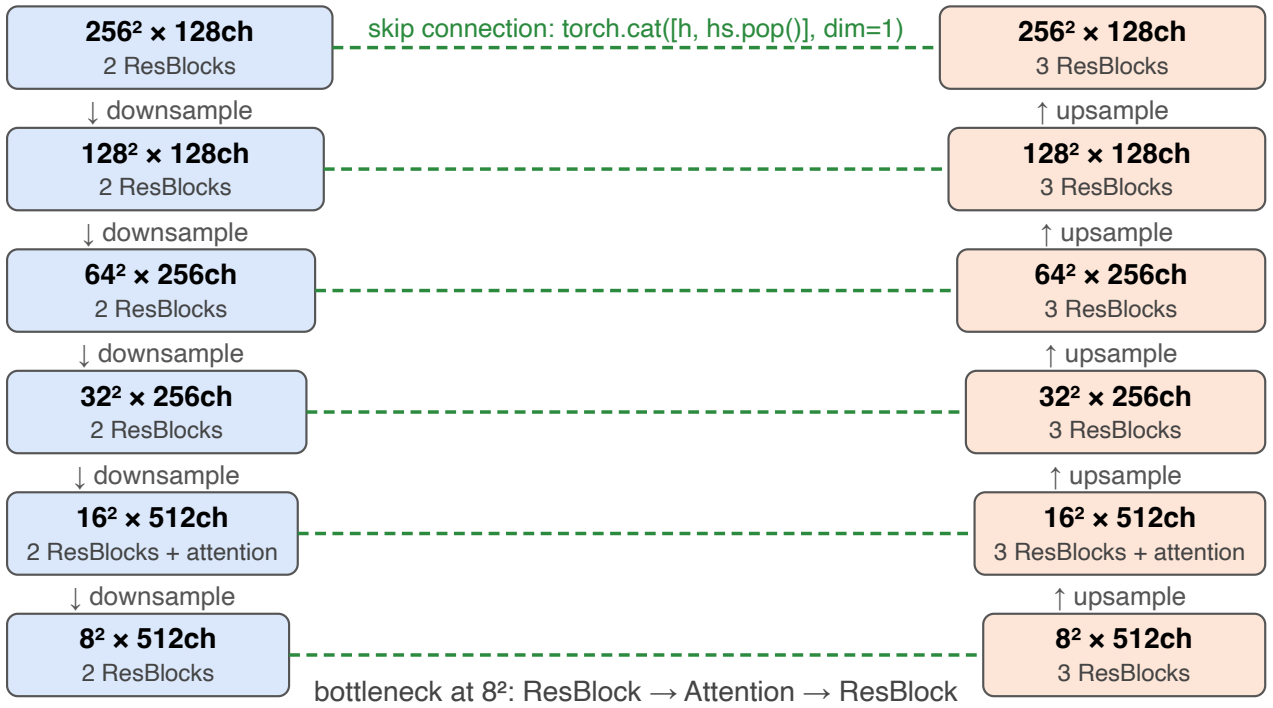
    h = self.norm2(h)
    h = nonlinearity(h)
    h = self.dropout(h)
    h = self.conv2(h)

    if self.in_channels != self.out_channels:
        x = self.nin_shortcut(x)   (2)
    return x + h   (3)
```

1. The time embedding is projected to one number per channel and **added to every pixel** of the feature map. This is how one network behaves differently at different noise levels.
2. A 1×1 convolution matches channel counts when they differ.
3. Residual connection: the block learns a correction to its input.

Normalization is GroupNorm with 32 groups. Dropout is 0.0 in the configs.

The U-Net




```
# downsampling: save every output for later
hs = [self.conv_in(x)]
...
# upsampling: concatenate the saved features
h = self.up[i_level].block[i_block](
    torch.cat([h, hs.pop()], dim=1), temb)   1
```

1. Skip connection: fine detail from the way down is handed back on the way up. Every ResBlock also receives `temb`.

Widths and sizes derived from the config (`ch=128`, `ch_mult=[1,1,2,2,4,4]`, `attn_resolutions=[16]`, 2 blocks per level) and the code structure; not printed from a run.

Conditioning: stack the condition with the noisy target

```
output = model(torch.cat([x_img, x], dim=1), t.float())  ❶
```

❶ Concatenate along the channel axis (`dim=1`), then run the U-Net.

> The condition is never noised. The network sees it clean at every step and learns to use it as a guide.

x_img (LD)
B × 1 × 256 × 256

+

x_t (noisy FD)
B × 1 × 256 × 256

→

U-Net input
B × 2 × 256 × 256

→

predicted ε
B × 1 × 256 × 256

model.type	loss	input channels	tasks
`sg`	sg_noise_estimation_loss	2 = 1 condition + 1 noisy target	LDFDCT, BRATS
`sr`	sr_noise_estimation_loss	3 = 2 neighbouring slices + 1 noisy target	PMUB
`simple`	noise_estimation_loss	1 (unconditional)	baseline

`functions/losses.py (loss_registry); configs/*.yml (in_channels)`

The training objective

$$L(\theta) = E_{t,\,(x_0,\,c),\,\varepsilon} \| \varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon\,,\, c\,,\, t\,) \|^2$$

```
return (e - output).square().sum(dim=(1, 2, 3)).mean(dim=0)   1
```

1. Squared error between true noise `e` and predicted noise `output`, **summed** over channels and pixels, then averaged over the batch. (Paper Eq. 9.)

- Plain regression: no discriminator, no adversarial balance. This is the paper's argument for training stability versus GANs.
- Every chosen t has equal weight.
- Because the loss is a *sum*, its absolute value scales with image size; compare loss curves only at the same resolution.

Training loop

DRAFT

```
x_img = x['LD'].to(self.device)
x_gt  = x['FD'].to(self.device)
e = torch.randn_like(x_gt)   (1)
...  # choose t (previous slides)
loss = loss_registry[config.model.type](model, x_img, x_gt, t, e, b)
optimizer.zero_grad()
loss.backward()   (2)
optimizer.step()
if self.config.model.ema:
    ema_helper.update(model)   (3)
if step % self.config.training.snapshot_freq == 0 or step == 1:
    states = [model.state_dict(), optimizer.state_dict(), epoch, step]
    if self.config.model.ema:
        states.append(ema_helper.state_dict())
    torch.save(states, os.path.join(
        self.args.log_path, "ckpt_{}.pth".format(step)))
    torch.save(states, os.path.join(self.args.log_path, "ckpt.pth"))   (4)
```

1. Fresh random noise every step.
2. Standard backprop; optimizer is Adam (`functions/__init__.py`).
3. Update the moving-average copy of the weights (next slides).
4. A checkpoint at step 1 and every `snapshot_freq` steps; `ckpt.pth` always holds the latest and is what `--resume_training` reads.

Loop ends only when `n_epochs` is reached; `n_iters` in the config is not checked (see “To verify”).

EMA: a smoothed copy of the weights

$$\theta_{EMA} \leftarrow (1-\mu)\cdot\theta + \mu\cdot\theta_{EMA}, \quad \mu = 0.999$$

```
def update(self, module):
    ...
    for name, param in module.named_parameters():
        if param.requires_grad:
            self.shadow[name].data = ((1. - self.mu) * param.data
                + self.mu * self.shadow[name].data)   (1)

def ema(self, module):      # copy shadow weights INTO the model
    ...
    param.data.copy_(self.shadow[name].data)   (2)
```

1. Each step moves the shadow copy 0.1% toward the current weights, so it averages roughly the last 1,000 updates.
2. At sampling time `ema(model)` overwrites the model with the smoothed weights. **Generated images come from the EMA weights, not the raw ones.**

Why: single-step weights are noisy; the average gives steadier, usually better samples. The paper does not discuss EMA; it comes with the DDIM codebase.

Saving and loading models

```
# saved (training):
states = [model.state_dict(), optimizer.state_dict(),
          epoch, step, ema_state]

# loaded (sampling):
states = torch.load(
    os.path.join(self.args.log_path, f"ckpt_{ckpt_idx}.pth"),
    map_location=self.config.device)  (1)
model.load_state_dict(states[0], strict=True)  (2)
ema_helper.load_state_dict(states[-1])  (3)
ema_helper.ema(model)  (4)
```

1. Path = `<exp>/logs/<doc>/ckpt_<N>.pth`, with N taken from the list `sampling.ckpt_id` in the config.
2. Index 0: the raw model weights.
3. Last index: the EMA weights.
4. Overwrite the model with the EMA weights, then sample.

> Practical consequence: the authors' single file `ckpt_LDFDCT.pth` only loads if it is renamed `ckpt_1.pth` and the config sets `ckpt_id: [1]`. The `--use_pretrained` flag does *not* do this; it targets unrelated CIFAR/LSUN models.

Sampling: the DDIM step

$$\hat{x}_0 = (x_t - \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon_\theta) / \sqrt{\bar{\alpha}_t} \quad \rightarrow \quad x_{t'} = \sqrt{\bar{\alpha}_{t'}} \cdot \hat{x}_0 + \sqrt{(1-\bar{\alpha}_{t'}-\sigma^2)} \cdot \varepsilon_\theta + \sigma\, z$$

```
for i, j in zip(reversed(seq), reversed(seq_next)):   (1)
    at      = compute_alpha(b, t.long())
    at_next = compute_alpha(b, next_t.long())   (2)
    xt = xs[-1].to('cuda')
    et = model(torch.cat([x_img, xt], dim=1), t)   (3)
    x0_t = (xt - et * (1 - at).sqrt()) / at.sqrt()   (4)
    c1 = kwargs.get("eta", 0) * (
        (1 - at / at_next) * (1 - at_next) / (1 - at)).sqrt()    # σ
    c2 = ((1 - at_next) - c1 ** 2).sqrt()
    xt_next = (at_next.sqrt() * x0_t
               + c1 * torch.randn_like(x) + c2 * et)   (5)
```

1. Walk the chosen time steps from the noisiest to 0 (about 10 iterations). `seq_next` is the same list shifted by one, ending with –1.
2. `compute_alpha` prepends a zero to β, so t = –1 gives ᾱ = 1: the last step lands on a fully clean image.
3. The same U-Net call as in training: condition stacked with the current noisy image.
4. Estimate the clean image from the predicted noise.
5. Re-noise $\hat{x}_0$ to the next level. `eta=0` (default): σ = 0, fully **deterministic**.

Evaluation, and our result

```
x_gt = x_gt*255
x = x*255
psnr_temp = calculate_psnr(x[i,:,:], x_gt[i,:,:])   (1)
ssim_temp = ssim(x_gt[i,:,:], x[i,:,:], data_range=255)   (2)
...
logging.info('Average: PSNR {}, SSIM {}, time {}'.format(
    avg_psnr, avg_ssim, avg_time))
```

(1) PSNR = $20 \cdot \log_{10}(255 / \sqrt{\text{MSE}})$, in dB. Higher is better.

(2) SSIM from scikit-image: compares local mean, variance and covariance; 1.0 is identical.

LDFDCT, 3,501 test images	PSNR	SSIM
Paper, Table II (Fast-DDPM)	37.5	0.92
Ours (pretrained weights, Easley L40S, PyTorch 2.2.2)	**37.46**	**0.9155**
Paper, DDPM (1000 steps)	35.4	0.87

> Our run uses the authors' trained checkpoint, so it validates the pipeline (data, sampler, metrics) but **not** training. It matched despite a different GPU and PyTorch version.

Each log line is a batch of 16 images; the final line averages all 3,501.

Config keys that control behavior

Key (configs/ldfd_linear.yml)	Value	Effect
`data.train_dataroot / sample_dataroot`	paths	Where images are read. We use absolute scratch paths.
`data.image_size`	256	Slices are resized to this; U-Net asserts square input of this size.
`model.type`	sg	Selects the loss and the input-channel layout.
`model.in_channels / out_ch`	2 / 1	Condition + noisy target in; predicted noise out.
`model.ch, ch_mult, num_res_blocks`	128, [1,1,2,2,4,4], 2	U-Net width and depth.
`model.attn_resolutions`	[16]	Self-attention only at 16×16 feature maps.
`model.ema_rate`	0.999	EMA smoothing μ.
`diffusion.beta_*, num_diffusion_timesteps`	1e-4, 0.02, 1000	The underlying noise curve.
`training.batch_size`	16	Matches the paper.
`training.snapshot_freq`	100000	Checkpoint interval (we use 10 for smoke tests).
`sampling.ckpt_id`	list	Which `ckpt_N.pth` files to load; we use `[1]`.
`sampling_fid.batch_size`	128 → **16**	128 runs out of memory on an L40S (44 GB); the paper used an 80 GB A100.
`optim.lr`	2e-5	Paper says 2e-4 (see “To verify”).

configs/ldfd_linear.yml; our changes in configs/ldfd_smoke_test.yml

Running it on the cluster: the job script

```
#!/bin/bash
#SBATCH --job-name=fastddpm-ldfdct-smoke   (1)
#SBATCH --partition=interactive   (2)
#SBATCH --gres=gpu:l40s:1   (3)
#SBATCH --cpus-per-task=8   (4)
#SBATCH --mem=32G
#SBATCH --time=01:00:00   (5)
#SBATCH --output=fastddpm-ldfdct-smoke-%j.out   (6)
#SBATCH --error=fastddpm-ldfdct-smoke-%j.err

source ~/.bashrc
conda activate fastddpm   (7)
cd ~/ECE_551_problems_ML/fast-ddpm   (8)
python fast_ddpm_main.py \
  --config ldfd_smoke_test.yml --dataset LDFDCT \
  --exp ~/carc-scratch/fastddpm-exp --doc ldfdct_smoke_test \
  --sample --fid --scheduler_type uniform --timesteps 10
```

1. Name shown in `squeue`.
2. Queue: `interactive` (4 h cap) for tests; `l40s` or `h100` (2 days) for long runs. `general` has no GPUs.
3. One L40S GPU.
4. CPU cores for data loading (matches `num_workers: 8`).
5. Walltime limit: the scheduler kills the job at this time.
6. `%j` = job id. Python's logging and progress bars go to the **.err** file; .out stays empty.
7. Activate the environment inside the job.
8. Configs and data paths are relative to this folder, so the job must start here.

slurm/ldfdct_smoke_test.sh

What running on HPC taught us

- **GPU memory sets batch size.** The paper's sampling batch of 128 (80 GB A100) fails with *CUDA out of memory* on an L40S; 16 works. Settings do not transfer between GPUs.
- **Software must match the GPU.** The paper's PyTorch 1.12.1 has no L40S/H100 support, so we use 2.2.2, which in turn forced `numpy<2` and `opencv-python<4.10`.
- **Installs belong on the login node.** Installing inside a GPU allocation burned walltime and was killed at the limit.
- **Data lives on scratch, not in git.** `~/carc-scratch`: 7.3 GB dataset bundle, 1.7 GB checkpoint, run outputs.
- **Read the .err file.** A failed batch job shows nothing at submit time; the traceback is in `.err`.
- **Training can delete a folder.** A fresh (non-resume) training run removes an existing `--doc` directory, so training and inference need different `--doc` names.
- **The README was incomplete.** No `requirements.txt`; five imports (`yaml, pandas, lmdb, matplotlib, requests`) were undocumented; one version pairing was invalid.
- **Cancel jobs you no longer need.** GPUs are shared.

To verify

- **Learning rate.** Paper: 2×10^{-4}. Configs: `lr: 0.00002` (2×10^{-5}). Which is intended?
- **Uniform scheduler length.** `arange(-1, 1000, 100)` with the first set to 0 gives 11 values, not 10.
- **Iteration cap.** `n_iters` is never checked in the training loop; the paper's 400,000 iterations are not enforced.
- **Validation.** `validation_freq` is effectively disabled and no validation loop was found; only training loss is logged to TensorBoard.
- **Multi-GPU.** Training uses `DataParallel`, not `DistributedDataParallel`.
- **Training behavior on our stack.** Inference matched the paper; whether training from scratch on PyTorch 2.2.2 / L40S matches is *empty* (not yet run).
- **Textbook references.** Chapter and equation numbers: *empty*.

From 2D to 3D: what would change

- **Layers:** 2D convolutions, normalization and attention become 3D (`Conv2d → Conv3d`); the U-Net data path stays the same.
- **Data:** loaders return volumes or sub-volumes (crops), shape `B × C × D × H × W`, not slices.
- **Memory:** activations grow roughly with the number of voxels, about D times a 2D slice. At the same batch size this multiplies memory by D, so batch size or crop size must shrink.
- **Compute:** per-step cost also grows about $D\times$; this is why the paper expects 3D diffusion to take “weeks to months.”
- **Paper's proposal:** run diffusion in a smaller 3D latent space learned by an autoencoder.
- **Small-scale example:** the MLSP 2026 video-diffusion tutorial (Pattichis) trains a 3D U-Net on downsampled 16×32×32 clips.

> Analysis, not a measured result. Measuring seconds per step and GPU memory at several batch sizes on Easley would turn this into numbers.